

</>

The Anuj DECODED

</>

```
01 #include <iostream>
02 #include <vector>
03 #include <unordered_map>
04 using namespace std;
05
06 class Solution {
07 public:
08     vector<int> twoSum(
09         vector<int>& nums,
10         int target) {
11         unordered_map<int, int> mp;
12         for (int i = 0; i < nums.size(); ++i) {
13             int diff = target - nums[i];
14             if (mp.count(diff)) {
15                 return {mp[diff], i};
16             }
17             mp[nums[i]] = i;
18         }
19         return {};
20     }
};
```



```
01 // Think Deeper
02 // Write Cleaner
03 // Solve Smarter
04 // Get Stronger
05 // Every day
06 // a better
07 // problem solver
```

CODE. SOLVE. **IMPROVE**. REPEAT.



THINK
DEEPER



WRITE
CLEANER



SOLVE
SMARTER



GET
STRONGER

“

Problem today,
Stronger tomorrow.

This Notebook Belongs To:

ANUJ

3550 · Smallest Index With Digit Sum Equal to Index

LeetCode · Daily Question

Easy

Array

Math

1. What Type of Question Is This?

This is a small **per-element condition check** problem: scan an array, compute one simple derived value for each element (here, a digit sum), and find the first index where that derived value matches the index itself. The only real sub-skill is knowing how to compute a number's **digit sum** efficiently — everything else is a single straightforward left-to-right scan.

2. What Is Given?

- An integer array **nums**.

3. What Do We Need to Find?

The **smallest index i** such that the sum of the decimal digits of `nums[i]` equals `i` itself. If no such index exists, return **-1**.

Function Signature

■ C++

```
class Solution {
public:
    int smallestIndex(vector<int>& nums) {

    }
};
```

Examples Walkthrough

Example 1 — used as the running example for every code sample below

```
Input:  nums = [1,3,2]
Output: 2
```

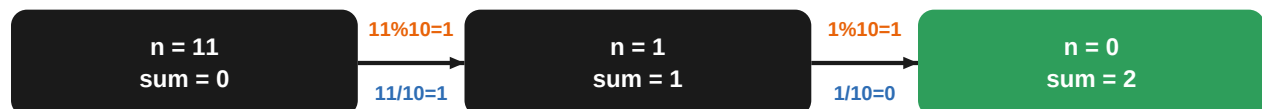
Index 0: $\text{digitSum}(1) = 1 \neq 0$. Index 1: $\text{digitSum}(3) = 3 \neq 1$. Index 2: $\text{digitSum}(2) = 2 = i \rightarrow$ matches, and since we scan left to right, this is automatically the smallest such index $\rightarrow 2$.

Example 2 — two-digit numbers

```
Input:  nums = [1,10,11]
Output: 1
```

Index 0: $\text{digitSum}(1) = 1 \neq 0$. Index 1: $\text{digitSum}(10) = 1 + 0 = 1 = i \rightarrow$ matches immediately. (Index 2 would also match, $\text{digitSum}(11) = 1 + 1 = 2 = i$, but since index 1 matched first, that's the answer) $\rightarrow 1$.

Computing $\text{digitSum}(11)$ by peeling off digits with % and /



$n == 0$, so we stop — $\text{digitSum}(11) = 2$, matching index $i = 1$

Figure 1 — peeling the last digit off with % 10, then discarding it with / 10, repeated until the number reaches 0.

Example 3 — nothing matches

```
Input:  nums = [1,2,3]
Output: -1
```

None of $\text{digitSum}(1)=1$, $\text{digitSum}(2)=2$, $\text{digitSum}(3)=3$ equal their respective indices 0, 1, 2 $\rightarrow -1$.

Constraints

- $1 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 1000$

Both bounds are tiny — this problem is about writing a correct, clean digit-sum helper and a straightforward scan, not about performance.

Overall Workflow & How to Think About It

Step-by-step reasoning to arrive at the right approach:

- **Step 1 — Write a digit-sum helper.** Repeatedly take a number modulo 10 to peel off its last digit, add that digit to a running sum, then divide the number by 10 (integer division) to discard that digit. Stop once the number reaches 0.
- **Step 2 — Scan the array left to right.** For each index i , compute `digitSum(nums[i])` using the helper and compare it to i .
- **Step 3 — Stop the moment you find a match.** Because you're scanning in increasing index order, the very first match you encounter is automatically the **smallest** valid index — there's no need to check the rest of the array once you've found one.
- **Step 4 — If the scan finishes with no match,** return -1.

Which Algorithm Is Best?

A single left-to-right pass, computing each digit sum with simple arithmetic and returning the instant a match is found, is $O(n \cdot d)$ time (d = number of digits, at most 4 here) and $O(1)$ space — already optimal, since every element must be examined at least once in the worst case (when the answer is -1). The tiers below show the difference between computing digit sums via string conversion versus direct arithmetic, and between a wasteful two-pass structure versus stopping as soon as the answer is known.

Approach 1 — Brute Force (String Conversion, Two Separate Passes)

Idea: Compute every element's digit sum up front by converting each number to a string and summing its characters, storing all the results in an array. Only afterward, in a second pass, scan for the first index where the stored digit sum matches the index.

Steps

- For each number, convert it to a string and sum the numeric value of each character to get its digit sum.
- Store every element's digit sum in an array, one full pass over nums.
- In a second pass, scan that array for the first index where the stored value equals the index.
- Return that index, or -1 if the second pass finds no match.

Approach 1 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int digitSumViaString(int x) {
        string str = to_string(x);          // convert to a string first
        int sum = 0;
        for (char c : str) sum += (c - '0'); // then sum the character digits
        return sum;
    }

    int smallestIndex(vector<int>& nums) {
        int n = nums.size();
        vector<int> sums(n);
        for (int i = 0; i < n; i++) sums[i] = digitSumViaString(nums[i]); // pass 1: compute all

        for (int i = 0; i < n; i++) {      // pass 2: scan for the first match
            if (sums[i] == i) return i;
        }
        return -1;
    }
};

int main() {
    vector<int> nums = {1, 3, 2};
    Solution sol;
    cout << sol.smallestIndex(nums) << endl; // Expected output: 2
    return 0;
}
```

■ Python

```
class Solution:
    def digitSumViaString(self, x):
        s = str(x)          # convert to a string first
        return sum(int(c) for c in s) # then sum the character digits

    def smallestIndex(self, nums):
        n = len(nums)
        sums = [self.digitSumViaString(x) for x in nums] # pass 1: compute all

        for i in range(n):      # pass 2: scan for the first match
            if sums[i] == i:
                return i
        return -1
```

```
if __name__ == "__main__":
    nums = [1, 3, 2]
    print(Solution().smallestIndex(nums))    # Expected output: 2
```

■ JavaScript

```
class Solution {
    digitSumViaString(x) {
        const s = String(x);           // convert to a string first
        let sum = 0;
        for (const c of s) sum += Number(c);    // then sum the character digits
        return sum;
    }

    smallestIndex(nums) {
        const n = nums.length;
        const sums = nums.map(x => this.digitSumViaString(x));    // pass 1: compute all

        for (let i = 0; i < n; i++) {           // pass 2: scan for the first match
            if (sums[i] === i) return i;
        }
        return -1;
    }
}

const nums = [1, 3, 2];
console.log(new Solution().smallestIndex(nums));    // Expected output: 2
```

Complexity: $O(n \cdot d)$ time (d = digits per number, at most 4 here) for computing all digit sums via string conversion, plus $O(n)$ for the second scan — $O(n \cdot d)$ overall, but with unnecessary string allocation overhead and a wasted second pass when the answer could often be found much earlier.

Approach 2 — Better (Arithmetic Digit Sum, Still Two Passes)

Idea: Replace the string-based digit sum with a direct arithmetic version (% and / by 10), removing the string allocation overhead — but still compute every element's digit sum up front before scanning for the answer.

Steps

- For each number, repeatedly take it modulo 10 to get the last digit, add it to a running sum, then integer-divide by 10 to drop that digit; stop once the number reaches 0.
- Store every element's digit sum in an array, one full pass over nums.
- In a second pass, scan that array for the first index where the stored value equals the index.
- Return that index, or -1 if no match is found.

Approach 2 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int digitSum(int x) {
        int sum = 0;
        while (x > 0) {           // peel off digits with arithmetic, no string needed
            sum += x % 10;
            x /= 10;
        }
        return sum;
    }

    int smallestIndex(vector<int>& nums) {
        int n = nums.size();
        vector<int> sums(n);
        for (int i = 0; i < n; i++) sums[i] = digitSum(nums[i]); // pass 1: compute all

        for (int i = 0; i < n; i++) { // pass 2: scan for the first match
            if (sums[i] == i) return i;
        }
        return -1;
    }
};

int main() {
    vector<int> nums = {1, 3, 2};
    Solution sol;
    cout << sol.smallestIndex(nums) << endl; // Expected output: 2
    return 0;
}
```

■ Python

```
class Solution:
    def digitSum(self, x):
        s = 0
        while x > 0:           # peel off digits with arithmetic, no string needed
            s += x % 10
            x //= 10
        return s

    def smallestIndex(self, nums):
        n = len(nums)
        sums = [self.digitSum(x) for x in nums] # pass 1: compute all
```

```

        for i in range(n):          # pass 2: scan for the first match
            if sums[i] == i:
                return i
        return -1

if __name__ == "__main__":
    nums = [1, 3, 2]
    print(Solution().smallestIndex(nums))    # Expected output: 2

```

■ JavaScript

```

class Solution {
    digitSum(x) {
        let s = 0;
        while (x > 0) {              // peel off digits with arithmetic, no string needed
            s += x % 10;
            x = Math.floor(x / 10);
        }
        return s;
    }

    smallestIndex(nums) {
        const n = nums.length;
        const sums = nums.map(x => this.digitSum(x));    // pass 1: compute all

        for (let i = 0; i < n; i++) {    // pass 2: scan for the first match
            if (sums[i] === i) return i;
        }
        return -1;
    }
}

const nums = [1, 3, 2];
console.log(new Solution().smallestIndex(nums));    // Expected output: 2

```

Complexity: Still $O(n \cdot d)$ time overall, but with a smaller constant factor (no string allocation per number). Still does unnecessary work in the worst case — every digit sum gets computed even if the answer is found at index 0.

Approach 3 — Optimal (Single Pass, Return the Instant a Match Is Found)

Idea: Fold everything into one pass. Compute each digit sum with the same arithmetic helper as Approach 2, but check it against the index immediately and return right away on a match — there's no reason to compute digit sums for elements you'll never even look at once the answer is already known.

Steps

- Walk the array once, index by index.
- At each index, compute the digit sum of that element directly.
- If it equals the current index, return that index immediately.
- If the loop finishes without ever matching, return -1.

Approach 3 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int digitSum(int x) {
        int sum = 0;
        while (x > 0) {
            sum += x % 10;
            x /= 10;
        }
        return sum;
    }

    int smallestIndex(vector<int>& nums) {
        for (int i = 0; i < (int)nums.size(); i++) {
            if (digitSum(nums[i]) == i) return i;    // return the INSTANT a match is found
        }
        return -1;
    }
};

int main() {
    vector<int> nums = {1, 3, 2};
    Solution sol;
    cout << sol.smallestIndex(nums) << endl;    // Expected output: 2
    return 0;
}
```

■ Python

```
class Solution:
    def digitSum(self, x):
        s = 0
        while x > 0:
            s += x % 10
            x //= 10
        return s

    def smallestIndex(self, nums):
        for i, x in enumerate(nums):
            if self.digitSum(x) == i:    # return the INSTANT a match is found
                return i
        return -1

if __name__ == "__main__":
```

```
nums = [1, 3, 2]
print(Solution().smallestIndex(nums))    # Expected output: 2
```

■ JavaScript

```
class Solution {
  digitSum(x) {
    let s = 0;
    while (x > 0) {
      s += x % 10;
      x = Math.floor(x / 10);
    }
    return s;
  }

  smallestIndex(nums) {
    for (let i = 0; i < nums.length; i++) {
      if (this.digitSum(nums[i]) === i) return i;    // return the INSTANT a match is found
    }
    return -1;
  }
}

const nums = [1, 3, 2];
console.log(new Solution().smallestIndex(nums));    // Expected output: 2
```

Complexity: $O(n \cdot d)$ time in the worst case (when the answer is -1 or found at the very last index), but often much faster in practice since the scan stops the moment a match is found — and $O(1)$ extra space, since no array of precomputed digit sums is ever stored.

Side-by-Side Comparison

1. Brute Force	String-based digit sum, precompute all, then scan	$O(n \cdot d)$	$O(n)$ for the stored sums
2. Better	Arithmetic digit sum, still precompute all, then scan	$O(n \cdot d)$	$O(n)$ for the stored sums
3. Optimal	Arithmetic digit sum, single pass, return on first match	$O(n \cdot d)$ worst case, often less	$O(1)$

Key Takeaways to Remember

- **Digit sum via % 10 and / 10** is a standard, reusable pattern — peel off the last digit with modulo, discard it with integer division, repeat until the number reaches 0. No string conversion needed.
- **"Find the smallest index satisfying some condition"** almost always means you can **return immediately** the moment you find a match during a left-to-right scan — there's no need to compute anything for later elements or to store results for a second pass.
- Combining "compute a per-element value" and "check a condition" into the **same loop**, rather than splitting them into separate passes, is a small but consistent habit that avoids unnecessary work and extra memory.
- All three tiers were validated against **Example 1** (`nums = [1,3,2] → 2`) across C++, Python, and JavaScript, and checked against Examples 2 and 3 (`nums = [1,10,11] → 1`, `nums = [1,2,3] → -1`) as well.